# Maîtrise des Structures Répétitives (Boucles), Preuve de Terminaison et Invariants

**Thème principal :** Algorithmique et Programmation Structurée

**Synthèse exécutive :** Ce cours pose les fondements indispensables de l'itération en programmation structurée. L'objectif est d'éliminer l'anti-pattern du 'copier-coller' pour traiter des ensembles de données de taille arbitraire. Il formalise le choix rigoureux entre boucles bornées (nombre de tours fixé en amont) et non bornées (arrêt sur condition dynamique), tout en fournissant les outils mathématiques d'assurance qualité logicielle : le 'variant de boucle' pour démontrer formellement l'absence de boucle infinie (terminaison), et 'l'invariant de boucle' pour garantir mathématiquement l'exactitude du résultat final produit (correction).

## Typologie des Structures Itératives et Critères de Choix

En programmation, répéter des instructions sans dupliquer le code est essentiel pour traiter des volumes de données variables. L'enjeu majeur est de savoir quel type de boucle choisir selon la situation. On distingue l'itération bornée (où le nombre de tours est déterminé à l'avance) et l'itération non bornée (où la fin dépend d'une condition dynamique). Le choix entre TANT QUE et RÉPÉTER repose exclusivement sur le nombre minimal d'exécutions requises (0 à N pour le TANT QUE, 1 à N pour le RÉPÉTER).

- Anti-pattern du Copier-Coller : Dupliquer du code le rend rigide, illisible et inmaintenable face à des volumes de données variables.
- Itération Bornée (POUR) : Structure employée lorsque le nombre d'itérations est fixe et connu au préalable. La variable de contrôle s'incrémente automatiquement.
- Itération Non Bornée à Pré-condition (TANT QUE) : La condition est évaluée avant le corps de boucle. Le traitement peut être exécuté 0 fois si la condition est fausse dès le début.
- Itération Non Bornée à Post-condition (RÉPÉTER... JUSQU'À) : Le corps de boucle est exécuté au moins une fois avant l'évaluation de la condition d'arrêt (1 à N exécutions).

***Cas concret enseignant :*** *L'analogie du coureur sur la piste d'athlétisme : Si l'entraîneur dit 'Fais 10 tours', c'est une boucle POUR. S'il dit 'Cours tant qu'il ne pleut pas', c'est un TANT QUE (s'il pleut déjà au vestiaire, le coureur ne court pas du tout = 0 tour). S'il dit 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est un RÉPÉTER... JUSQU'À (le coureur fera au moins un tour avant d'évaluer son état).*

## Preuve Formelle de Terminaison : Le Variant de Boucle

Pour démontrer formellement qu'une boucle non bornée s'arrête (preuve de terminaison), il faut identifier un 'variant de boucle'. Un variant V est une fonction entière qui exprime une quantité calculée sur les variables de la boucle. La preuve repose sur la propriété du bon ordre des entiers naturels : il n'existe pas de suite d'entiers strictement décroissante et minorée qui soit infinie. Si le variant décroît strictement et est minoré par une constante, la terminaison est scientifiquement garantie.

- Boucle Infinie : Dysfonctionnement critique provoquant le gel de l'application si la condition de sortie n'est jamais atteinte.
- Variant de Boucle : Grandeur entière relative à l'état du programme mesurant la 'distance' jusqu'à l'arrêt.
- Décroissance Stricte : Le variant doit diminuer d'au moins 1 entier à chaque tour de boucle ($V_{k+1} < V_k$).
- Minorant : Valeur plancher (généralement 0) en dessous de laquelle le variant ne peut pas descendre.

***Cas concret enseignant :*** *La métaphore du réservoir d'essence d'une voiture : À chaque kilomètre parcouru (itération), le volume d'essence dans le réservoir (le variant) diminue obligatoirement d'une quantité stricte. Comme le réservoir ne peut pas descendre sous 0 litre (minorant), la voiture finira obligatoirement par s'immobiliser. Si la jauge ne baissait pas à chaque kilomètre, la voiture roulerait à l'infini : c'est le bug de la boucle infinie.*

## Preuve Formelle de Correction : L'Invariant de Boucle

Savoir qu'une boucle s'arrête ne suffit pas : il faut prouver qu'elle calcule le bon résultat. C'est le rôle de l'invariant de boucle, qui fonctionne exactement comme un raisonnement par récurrence en trois étapes : l'initialisation, la conservation (hérédité) et la conclusion. À la sortie de la boucle, l'invariant combiné à la négation de la condition de boucle démontre mathématiquement la validité du résultat.

- Invariant de Boucle : Assertion logique (propriété) qui reste VRAIE avant, pendant et après chaque itération.
- Initialisation : Démonstration que l'invariant est vrai dès l'entrée dans la boucle.
- Conservation (Hérédité) : Démonstration que si l'invariant est vrai au début du tour k, il le reste à la fin du tour k.
- Correction (Conclusion) : Combinaison de l'invariant final et du test d'arrêt prouvant la validité du résultat.

***Cas concret enseignant :*** *L'exemple du livreur de journaux : Au départ de sa tournée, l'égalité 'Journaux distribués + Journaux restants dans le sac = Total initial' est vraie. À chaque maison servie, les deux quantités changent, mais leur somme reste rigoureusement la même (invariant). Quand le sac est vide à la fin de la tournée (condition de sortie), le nombre de journaux distribués est obligatoirement égal au total initial.*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Est-ce qu'on peut toujours remplacer une boucle POUR par une boucle TANT QUE ?
**Réponse du professeur :** Oui, toute boucle POUR peut se réécrire sous forme de boucle TANT QUE en initialisant explicitement le compteur avant la boucle, en testant la condition et en incrémentant le compteur à l'intérieur du corps. Cependant, la boucle POUR reste plus lisible et sécurisée lorsque le nombre d'itérations est connu.

**Question étudiant :** Si j'utilise une boucle TANT QUE avec la condition (i < N), est-ce que le corps s'exécute si i est déjà égal à N dès le début ?
**Réponse du professeur :** Non, car la condition (i < N) est évaluée avant d'entrer dans le corps. Si i est égal à N (ou supérieur), la condition est fausse dès le départ et le corps s'exécute 0 fois.

## ■ Exercice & Démonstration d'Application

**Énoncé :** Écrire un algorithme en pseudo-code permettant de calculer la somme des N premiers entiers pairs stricts (par exemple pour N=3, sommer 2 + 4 + 6 = 12). Justifier le choix de la boucle, rédiger le code algorithmique et formaliser le variant de boucle garantissant la terminaison.

**Résolution pas à pas :**
Étape 1 : Choix de la boucle. La valeur N est connue dès l'entrée en paramètre ; le nombre d'itérations est donc déterminé et borné à N. La boucle 'POUR' est la structure la plus adaptée et la plus sécurisée. Étape 2 : Initialisation des variables. somme (accumulateur) initialisé à 0, terme (premier entier pair strict) initialisé à 2, et i (variable de contrôle de boucle). Étape 3 : Écriture du pseudo-code : Algorithme SommePairs Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N par pas de 1 Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin Étape 4 : Démonstration du variant de boucle. Si l'on traduit cette itération sous forme de boucle tant que contrôlée par le compteur i, la quantité V = (N - i + 1) est un entier naturel. À chaque tour de boucle, i est incrémenté de 1, donc V diminue strictement de 1 ($V_{k+1} = V_k - 1$). Le minorant est 0 (obtenu quand $i = N + 1$).

**Résultat : Pour N = 3 : Tour 1 (i=1) : somme = 2, terme = 4. Tour 2 (i=2) : somme = 6, terme = 6. Tour 3 (i=3) : somme = 12, terme = 8. Sortie de boucle. La valeur affichée est 12. L'algorithme se termine de manière scientifiquement prouvée et produit le résultat correct.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Quelle est la différence fondamentale en mémoire et à l'exécution entre une boucle TANT QUE et une boucle RÉPÉTER... JUSQU'À ?**

*R : La boucle TANT QUE évalue sa condition avant l'exécution (0 à N tours possibles), tandis que la boucle RÉPÉTER... JUSQU'À l'évalue après le corps (1 à N tours garantis).*

**Q : Quelles sont les deux conditions mathématiques indispensables qu'une fonction entière doit vérifier pour constituer un variant de boucle valide ?**

*R : 1) Décroissance stricte à chaque itération ($V_{k+1} < V_k$), et 2) Minoration par une borne constante (généralement 0).*

**Q : Quel oubli classique dans l'écriture d'une boucle TANT QUE entraîne une boucle infinie et fait perdre l'intégralité des points en DS d'algorithmique ?**

*R : L'oubli de la mise à jour (l'incrémentation ou la modification) des variables de la condition de boucle à l'intérieur du corps, ce qui fait que la condition reste éternellement vraie.*